MICRO PROJECT – PROFESSIONAL PROJECT BRIEF

ACADEMIC RESOURCE SHARING HUB

A SaaS-Based Centralized Academic Resource Management Platform

Student Project Specification

Project Attribute	Specification
Project Type	Second-Year Micro Project / SaaS Web Application
Primary Users	Students
Administrative Users	Staff Members
Database	MySQL – Mandatory
Application Type	Role-Based Web Application
AI Requirement	No AI functionality required inside the application
Development Support	AI coding assistants may be used responsibly

1. Project Overview

Academic Resource Sharing Hub is a web-based SaaS platform designed to provide a centralized and organized repository for academic resources within a college. The platform enables students to discover, search, view, download, bookmark, request and contribute academic resources. Authorized staff members manage the repository, verify student contributions, organize resources, respond to resource requests and monitor platform activity.

The system follows a Student–Staff role model. Students are the primary users of academic resources, while staff members act as administrators who maintain the quality and organization of the repository.

The final product is a conventional SaaS application. Artificial Intelligence, Machine Learning, Generative AI and chatbot functionality are not required inside the application. AI coding tools may be used during development, but students remain responsible for understanding, testing and explaining their code.

2. Problem Statement

Academic resources are often distributed through WhatsApp groups, email, Google Drive folders, classroom groups, department websites, faculty sharing and physical copies. This makes it difficult for students to find the correct and latest material when needed.

Staff members also spend time repeatedly sharing resources, answering requests and organizing files across different locations. There is often no centralized mechanism to verify student-contributed resources or track resource requests.

The proposed platform provides a single, structured and role-based system for academic resource discovery, sharing, verification and management.

3. Project Objectives

- Centralize academic resources in one platform.
- Allow students to search and access resources based on academic requirements.
- Organize resources by department, year, semester, subject and resource type.
- Allow students to request resources that are not available.
- Allow students to contribute useful resources for staff verification.
- Allow staff to approve, reject, organize and manage resources.
- Provide role-based access for students and staff.
- Use MySQL as the mandatory relational database.
- Provide dashboards for students and staff.
- Track basic resource usage such as downloads.
- Reduce dependency on scattered communication channels.

4. Target Users and Roles

Role	Purpose	Main Access
Student	Discover and use academic resources	Search, view, download, bookmark, request and contribute
Staff / Admin	Manage and maintain the academic repository	Upload, verify, approve, reject, categorize, manage and monitor

5. Academic Resources Covered

- Lecture Notes
- Study Materials
- PowerPoint Presentations
- Previous-Year Question Papers
- Question Banks
- Laboratory Manuals
- Laboratory Reference Materials
- Syllabus and Course Documents
- Project Guidelines
- Project Reference Documents
- Seminar Materials
- Programming and Technical Resources
- Student-Contributed Academic Materials

6. Functional Modules

Module	Description
1. Authentication & Authorization	Student registration/login, staff login, logout, profile management, password security and role-based access.
2. Student Dashboard	Summary of resources, bookmarks, contributions and requests.
3. Resource Discovery	Browse the structured academic repository.
4. Search & Filtering	Search/filter by keyword, department, year, semester, subject, type and academic year.
5. Resource Details	Display complete metadata and actions for an individual resource.
6. Resource Upload	Allow authorized staff to publish academic resources.
7. Student Contribution	Allow students to submit resources for staff review.
8. Resource Verification	Staff reviews and approves/rejects submissions.
9. Resource Requests	Students request resources that are unavailable.
10. Bookmarks	Students save frequently used resources.
11. Download Tracking	Record resource downloads for basic statistics.
12. Staff Dashboard	Monitor resources, requests, contributions and statistics.
13. Category Management	Manage departments, subjects, resource types and academic years.
14. Status Management	Control Pending, Under Review, Verified, Published, Rejected and Archived states.

7. Student Features

- Register and log in securely.
- View a personalized dashboard.
- Browse the academic repository.
- Search resources by keyword.
- Filter resources by academic attributes.
- View resource details and metadata.

- Download permitted resources.
- Bookmark useful resources.
- Submit resource requests.
- Upload academic resources for staff verification.
- Track submitted-resource status.
- Track resource-request status.
- View contribution history.

8. Staff / Admin Features

- Secure staff login.
- View staff dashboard and platform statistics.
- Upload academic resources.
- Edit and archive resources.
- Review student-contributed resources.
- Approve or reject submissions.
- Add verification remarks.
- Respond to student resource requests.
- Manage departments.
- Manage subjects.
- Manage resource types.
- Manage academic years.
- Monitor downloads and basic usage.
- Remove outdated, duplicate or inappropriate resources.

9. Resource Verification Workflow

Student-contributed resources must not become immediately visible in the main repository.

Student Upload → PENDING

PENDING → UNDER REVIEW

UNDER REVIEW → VERIFIED / REJECTED

VERIFIED → PUBLISHED

Published Resource → ARCHIVED when no longer relevant

10. Resource Request Workflow

1. Student searches the repository.
2. If the required resource is unavailable, the student creates a resource request.
3. Staff views pending requests from the staff dashboard.
4. Staff provides or publishes a suitable resource.
5. The request is updated as completed or closed.
6. The student views the final request status.

11. Search and Filtering Requirements

Search and filtering are mandatory because students must be able to locate the correct resource quickly.

Filter	Example
Department	AI&DS
Year	II Year
Semester	III / IV
Subject	Database Management Systems
Resource Type	Question Paper
Academic Year	2026–27
Status	Published / Verified

12. Suggested Application Pages

Area	Pages
Public	Home, About, Login, Register
Student	Dashboard, Resources, Resource Details, Bookmarks, Resource Requests, My Contributions, Profile
Staff	Dashboard, Manage Resources, Verify Contributions, Resource Requests, Departments, Subjects, Resource Types, Reports, Profile

13. Recommended MySQL Database Design

Table	Key Fields / Purpose
users	id, name, email, password_hash, role, department_id, created_at
departments	id, name, created_at
subjects	id, name, department_id, year, semester, created_at
resource_types	id, name, created_at
resources	id, title, description, subject_id, resource_type_id, uploaded_by, file_path, academic_year, status, created_at
resource_requests	id, student_id, subject_id, title, description, status, staff_remarks, created_at
bookmarks	id, student_id, resource_id, created_at
resource_downloads	id, student_id, resource_id, downloaded_at
resource_reviews	id, resource_id, staff_id, status, remarks, reviewed_at

13.1 Basic Database Relationships

- One department can have many subjects.
- One user can upload many resources.
- One subject can have many resources.
- One resource type can be associated with many resources.
- One student can create many resource requests.
- One student can bookmark many resources.
- One resource can have many download records.
- Staff review records are associated with resources.

14. Technology Stack

Layer	Technology	Purpose
Frontend	React.js	User interface, dashboards, forms and resource browsing
Backend	Node.js + Express.js	REST APIs and business logic
Database	MySQL – Mandatory	Relational data storage
Authentication	JWT	Authentication and protected routes
Password Security	bcrypt	Secure password hashing
UI	Tailwind CSS or Bootstrap	Responsive interface
API Testing	Postman	API testing and debugging
Version Control	Git + GitHub	Source control and collaboration
Development	VS Code	Application development
Development Assistance	AI coding tools	Optional development support

15. Non-Functional Requirements

- Responsive and usable interface.
- Clear navigation and consistent UI.
- Validation for important forms.
- Passwords must never be stored in plain text.
- Students must not access staff-only routes or APIs.
- Uploaded files must be validated for type and size.
- Proper primary and foreign keys must be used.
- Meaningful error and success messages.
- Organized and maintainable code.
- Installation and usage documentation.

16. Mandatory Features

Category	Mandatory Features
Authentication	Student registration/login, Staff login, logout, role-based access
Resources	Upload, view, edit/archive, download
Discovery	Search, filters and categories
Student Contribution	Submit resource and track status
Verification	Staff review, approve/reject and remarks
Requests	Student resource request and staff response/status
Bookmarks	Add/remove bookmark and bookmark list
Dashboard	Student and staff dashboards
Database	MySQL with proper relationships
Security	Password hashing, protected routes and basic file validation

17. Scope Restrictions

The following are not required for this micro-project:

- Artificial Intelligence or Machine Learning features
- Generative AI inside the application
- AI chatbot

- AI recommendation engine
- OCR or automatic document summarization
- Face recognition
- Real-time chat
- Payment gateway
- Blockchain
- Microservices architecture
- Native mobile application
- Complex external integrations

18. Optional Features

- Email notifications
- PDF preview
- Advanced analytics charts
- Resource version history
- Most-downloaded resources
- Recently added resources
- Resource ratings/feedback
- Advanced access permissions
- Export reports
- Multi-college support

19. Expected Development Workflow

7. Understand the problem and finalize requirements.
8. Design the database and relationships.
9. Create backend and REST API structure.
10. Implement authentication and role-based authorization.
11. Develop resource management APIs.
12. Develop contribution and verification workflow.
13. Develop search and filtering.
14. Develop student and staff dashboards.
15. Connect React frontend to backend APIs.
16. Implement validation and security controls.
17. Test major workflows.
18. Prepare documentation and demonstration.

20. Expected Deliverables

Deliverable	Requirement
Source Code	Complete React frontend and Node.js/Express backend
MySQL Database	SQL schema/script with tables, keys and relationships
README	Project overview, installation, configuration and run instructions
API Documentation	Major endpoints and their purpose
Screenshots	Major student and staff screens
Project Report	Problem, objectives, requirements, design, implementation,

	testing and conclusion
Presentation	Problem, solution, architecture, modules and demonstration
Working Application	Application that can be executed and demonstrated successfully

21. Suggested Evaluation Areas

Area	What Will Be Evaluated
Problem Understanding	Clarity of problem and solution
UI/UX	Navigation, responsiveness and usability
Frontend	React implementation, forms and API integration
Backend	REST APIs, business logic and error handling
MySQL	Schema, relationships, queries and data integrity
Authentication	Login, password security and RBAC
Core Workflow	Search, upload, contribution, verification and requests
Testing	Functional testing and error handling
Documentation	README, report, database and API documentation
Demonstration	Ability to explain and demonstrate the application

22. Student Guidelines for AI Coding Tools

- AI coding tools may be used to generate, explain, refactor or debug code.
- Students must understand the code before including it in the final project.
- AI-generated code must be tested and validated.
- Do not copy features blindly without understanding them.
- Never expose passwords, database credentials or API secrets to AI tools.
- Every team member should be able to explain the database, major APIs and major frontend components.
- AI assistance does not replace responsibility for the final application.

23. Expected Learning Outcomes

- Understand SaaS architecture.
- Develop a responsive React frontend.
- Build REST APIs using Node.js and Express.js.
- Design a relational MySQL database.
- Implement CRUD operations.
- Implement authentication and Role-Based Access Control.
- Handle file uploads and downloads.
- Implement search and filtering.
- Build verification and approval workflows.
- Develop dashboards and basic reports.
- Use Git and GitHub effectively.
- Use AI coding assistants responsibly while validating generated code.

24. Final Project Definition

Academic Resource Sharing Hub is a role-based SaaS platform that centralizes academic resources for a college. Students can discover, access, request and contribute resources, while staff members verify, organize, publish and manage the repository. The system uses React.js, Node.js, Express.js and MySQL with secure authentication and role-based access.

Build a useful, secure and understandable application — not just a collection of screens.